

AstroFixxer: An Offline Smartphone Push-To Guide with On-Device Plate Solving for Small Telescopes

[Author name]
[Department], [Institution]
[City], [Country]
kushmodi13@gmail.com

Abstract—Beginners with manual Dobsonian and small alt-azimuth telescopes often cannot find faint objects, because the mount gives no readout of where the tube points. Commercial answers add encoders or a camera dock, and the free phone apps that replace them rely on the phone’s compass and gyroscope, which drift and are disturbed by a metal tube. This paper reviews the free and commercial tools in this space and describes AstroFixxer, an open-source Android application that guides a telescope from a phone strapped to it. AstroFixxer extends the AstroHopper web application with a Stellarium-derived offline catalogue of 93,997 deep-sky objects, 8,913 stars and the IAU constellation boundaries, a star-alignment step, and a plate solver written in pure Kotlin that runs on the phone against 579,984 stars to magnitude 10.5 from Gaia DR3 and Hipparcos. The previous sidereal-time formula left pointing errors of up to 801 arcseconds against the astropy reference; the corrected precession model reduces the worst of 66 reference cases to 28 arcseconds. On synthetic star fields the solver solved all 29 test images, including 1-degree eyepiece fields to within 0.5 arcseconds, and refused all 28 negative cases, among them random-star fields. The application has not yet been validated with a real telescope, and the paper states which parts remain to be tested in the field.

Index Terms—amateur astronomy, push-to navigation, smartphone sensors, plate solving, star identification, offline mobile application

I. INTRODUCTION

A 150 mm or 200 mm Dobsonian telescope shows hundreds of galaxies and clusters, yet many first-time owners see only the Moon and the planets. The mount is two bearings and a tube. It has no setting circles, the finder shows a small patch of sky, and most deep-sky objects are too faint to see in it. Observers traditionally reach them by star hopping from a bright star with a paper chart, a skill that takes many nights to learn.

Three kinds of tools address this. Digital setting circles fit encoders to both axes and report the tube’s altitude and azimuth to a handset or a planetarium app such as SkySafari [1]. Camera-based finders photograph the sky and identify the star pattern: Celestron’s StarSense Explorer does this with the phone’s own camera looking into a mirror dock [2], and the open-source PiFinder uses a Raspberry Pi camera [3]. The third group uses a phone strapped to the tube as the sensor. AstroHopper [4] and SkEye [5] read the phone’s gyroscope, gravity sensor and compass, and ask the observer to point at one known star so that the app can remove the compass offset. This last approach needs no extra hardware, but the compass is

unreliable next to a steel tube and the gyroscope drifts within minutes, so the observer has to realign often.

AstroFixxer started as a web port of AstroHopper and is now a native Android application. It keeps the free, sensor-based design and adds the pieces that the review in Section II finds missing from the free tools: a large offline catalogue, an explicit and correctable alignment step, and camera plate solving that runs on the phone without a network connection or a dock.

The contributions described in this paper are:

- a review of free and commercial push-to tools and plate solvers, compared by the hardware they need, how they determine pointing, and whether they work offline (Section II, Table I);
- an offline data pipeline that converts Stellarium’s catalogues into compact Android assets, including 579,984 Gaia- and Hipparcos-based stars for plate solving (Section III-C);
- a corrected coordinate model whose worst error against astropy falls from 801 to 28 arcseconds over 66 reference cases (Section III-D);
- a pure-Kotlin plate solver with a statistical acceptance test, evaluated on synthetic wide-field and eyepiece images (Section III-F).

Section II reviews related software and algorithms. Section III describes what AstroFixxer implements, Section IV shows its design in UML diagrams, and Section V compares it with the reviewed tools and lists what has not yet been tested. Section VI concludes.

II. RELATED WORK

A. Phone-sensor push-to applications

AstroHopper by Artyom Beilis is a free progressive web application released under the GPLv3 [4]. Its author describes it as digital setting circles implemented in a smartphone. The phone is attached to the tube with its top edge pointing along the optical axis. The observer centres a known star, presses Align, waits about three seconds for the sensors to settle, and then moves the telescope until the displayed altitude and azimuth differences reach zero. The app needs a gyroscope and a gravity sensor; the compass is optional, and a manual azimuth mode covers phones without one. It works offline once cached and offers watch lists, a night mode and user-defined

objects. AstroFixxer is a derivative of this code and keeps its licence.

SkEye is an Android planetarium with a push-to mode [5]. In its “indirect mode” the observer points the telescope at a known object and tells the app what it is; the app stores the angular offset between the phone and the tube and then follows the phone’s compass, accelerometer and gyroscope. SkEye comes in a free Basic and a paid Pro version.

Both apps share the same weak point. The alignment offset is correct only while the sensors stay calibrated, and a magnetometer next to a steel tube or a metal mount is easily disturbed. Neither app can check its own pointing against the sky.

Sky Map (formerly Google Sky Map) is an open-source Android sky viewer that uses the same sensors to show the part of the sky the phone points at [6]. It shipped on the first Android phones, was open-sourced in 2012, and the current rewrite is GPLv3. It is not designed to be mounted on a telescope, but it is the best-known free example of sensor-driven sky display on Android.

B. Camera-based pointing

Celestron’s StarSense Explorer holds the phone in a dock above a small mirror, so the rear camera sees the sky the telescope points at [2]. The companion app plate-solves the camera image with a “lost in space” algorithm to find where the telescope points. Celestron says this was the first app to use plate solving for this purpose and that the technology is patented. The app works only with the StarSense Explorer telescopes that include the dock.

PiFinder is open-source hardware and software built around a Raspberry Pi, the Raspberry Pi HQ camera and a small display [3]. It solves the sky image up to 20 times a second and uses an accelerometer between exposures to update the chart and the push-to directions. It needs no alignment and no encoders, and it offers a catalogue of about 150,000 objects. Its plate solving uses the Cedar Detect and Cedar Solve libraries, which descend from tetra3.

These two systems show what a camera adds: the pointing is measured, not inferred from drifting sensors. Both need dedicated hardware: a dock tied to one telescope range, or a separate computer and camera.

C. Planetarium software with telescope control

Stellarium is a GPL desktop planetarium used both by amateurs and in cultural-astronomy research [7]. It includes a telescope control plugin for motorised mounts, and its catalogues, sky cultures and star files are published with the source. AstroFixxer takes all of its offline sky data from Stellarium. SkySafari is a commercial planetarium for phones and tablets [1]. It connects to digital setting circle encoders and GoTo mounts, draws the telescope’s field of view on the chart, and relies on the encoders for pointing.

D. Plate-solving and star-identification algorithms

Astrometry.net solves an image with no prior knowledge of where it points or of its scale [8]. It detects sources, forms

geometric hashes of groups of four or five stars, looks them up in pre-built index files, and accepts a hypothesis only if it passes a Bayesian decision test against the null hypothesis that the matches are chance. Covering a wide range of image scales needs large index files, which suits a desktop or a server better than a phone.

Tetra identifies stars from a single database access by storing star patterns in a directly addressed hash table [9]. It was designed for spacecraft star trackers, where the field of view is fixed and known, and it won the student competition at the 2017 Small Satellite Conference. Its open-source Python descendant tetra3 underlies the libraries used by PiFinder.

ASTAP is a free, open-source stacking program with a fast local plate solver [10]. It solves FITS, raw and common image formats on desktop systems and the Raspberry Pi against downloadable star databases, which are offered at several star densities.

Marbel, Ben-Moshe and Yozevitch showed that a star tracker can run on a commercial smartphone and recover the phone’s global orientation from its own camera [11]. Their target was the orientation of drones and nano-satellites, not telescope pointing, but the work shows that phone cameras detect enough stars for pattern matching.

E. Summary

Table I compares the tools. The free phone apps need no hardware but cannot verify their pointing. The camera systems measure pointing but need a dock or a separate computer. The solvers are general tools for desktops, servers or spacecraft, not for a phone at the eyepiece. No free tool in this review combines sensor push-to, camera plate solving on the phone itself, and offline operation. That gap is the one AstroFixxer targets.

III. WHAT ASTROFIXXER IMPLEMENTS

A. Origin, licence and scope

AstroFixxer began as a web application derived from AstroHopper [4], [12]. Because AstroHopper is GPLv3, the Android port is GPLv3 as well, and its source is public [13]. The Android version keeps AstroHopper’s pointing mathematics and adds an offline catalogue, events, a voice assistant and plate solving. It targets Android 8.0 (API level 26) and later.

B. Architecture

The app is written in Kotlin with Jetpack Compose. It is a single Gradle module with three layers. The astronomy layer (`astro/`) holds coordinates, catalogues, events, satellite passes and the plate solver, and it has no Android imports, so it runs under plain JVM unit tests. The interface layer (`ui/`) holds the screen state and the Compose screens; it also has no Android imports, which lets the same code run in a Compose Desktop test harness. Only `MainActivity` touches Android: sensors, location, speech, preferences and asset loading. The project deliberately avoids a dependency-injection framework, a database and a networking library, since the app ships everything it needs in its assets. The one

TABLE I
PUSH-TO TOOLS AND PLATE SOLVERS REVIEWED IN SECTION II

Tool	Platform	Pointing from	Extra hardware	Offline	Source
AstroHopper [4]	Web app (PWA)	Phone sensors, one-star alignment	None	Yes, once cached	GPLv3
SkEye [5]	Android	Phone sensors, object offset	None	Yes	Not published
Sky Map [6]	Android	Phone sensors (sky viewer only)	None	Yes	Apache 2.0 / GPLv3
StarSense Explorer [2]	iOS, Android	Phone camera, plate solving	Dock and mirror	Yes	Proprietary, patented
PiFinder [3]	Raspberry Pi	Camera plate solving + accelerometer	Pi, camera, display	Yes	Open source
SkySafari [1]	iOS, Android	Encoders or GoTo mount	Encoders or mount	Yes	Proprietary
Stellarium [7]	Desktop	Mount control plugin	Motorised mount	Yes	GPL
Astrometry.net [8]	Server, desktop	Blind plate solving	Camera	With local index	Open source
ASTAP [10]	Desktop, Pi	Plate solving	Camera	Yes	Open source
AstroFixxer (this work)	Android	Phone sensors, star alignment, on-phone plate solving	None	Yes	GPLv3

exception is an optional download of satellite orbit elements, which the app refreshes when a network is present. Section IV shows the structure in UML diagrams.

C. Offline data

A Python importer converts Stellarium’s data files into the app’s assets at build time. The main catalogue holds 93,997 deep-sky objects from Stellarium’s catalogue and 8,913 stars from the HYG v3 database [14] with their B–V colour indices. It also holds the modern and Indian sky cultures and 781 IAU constellation boundary edges. The boundaries are published for the 1875 equinox, so the importer precesses them to J2000 with the IAU 1976 model [15]; the result agrees with astropy [16] to 0.37 arcseconds. Comet and asteroid orbital elements and meteor shower data also come from Stellarium, and eclipses come from NASA’s published tables.

For plate solving, the importer decodes Stellarium’s binary star catalogues, which are built from Gaia DR3 [17] and Hipparcos [18]. It propagates each star from the catalogue epoch to J2000 using its proper motion and keeps 579,984 stars brighter than magnitude 10.5. Each star is stored in 5 bytes, sorted into declination bands and right-ascension cells so that a cone around a pointing can be read without scanning the file. The asset is 2.94 MB. Cross-matched against HYG, the 4,969 stars brighter than magnitude 6 have a median position difference of 0.046 arcseconds. Figure 1 shows what the two assets hold. Galaxies are 90.6% of the deep-sky objects, and 78% of the solver stars lie between magnitude 9 and 10.5.

D. Coordinate model

Planet positions come from VSOP87 [19] with light time, aberration, precession and a truncated IAU 2000B nutation series, ported from the web application. Satellite passes use SGP4 [20].

The web code converted catalogue positions to the local horizon with the Earth rotation angle applied to J2000 coordinates and no precession. Because the Earth rotation angle omits the accumulated precession in right ascension

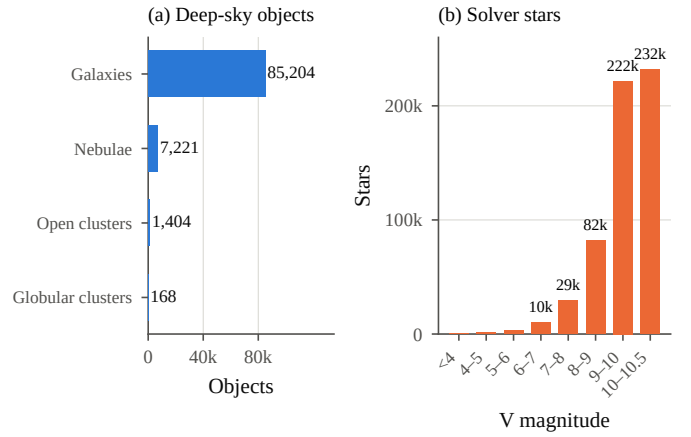


Fig. 1. (a) Deep-sky objects by type in `sky_catalog.json.gz`, 93,997 in all. (b) Stars per magnitude bin in `solver_stars.bin`, 579,984 in all

that Greenwich sidereal time contains, this formula happens to cancel most of the precession in right ascension. The part that does not cancel, the change in declination, grows by about 20 arcseconds a year. We measured it against astropy for 12 bright stars on 4 dates and at 3 sites (New Delhi, Sydney and Tromsø), which gives 66 cases above 5 degrees altitude. The worst error was 801 arcseconds. Figure 2 plots the error of every case against the date. The error of the old formula grows with the years since J2000, while at J2000 itself the two formulas agree.

The corrected model applies each term once. A J2000 unit vector $\mathbf{r}_0$ is rotated to the mean equator of date with the IAU 1976 angles [15],

$$\mathbf{r} = R_3(-z) R_2(\theta) R_3(-\zeta) \mathbf{r}_0, \quad (1)$$

and the hour angle is taken from Greenwich mean sidereal time, $h = \text{GMST} + \lambda - \alpha$, where λ is the observer’s east longitude and α the right ascension of date. The inverse transformation, used to report where the telescope points, applies the transpose. Nutation (up to 17 arcseconds) and annual aberration (up to 20 arcseconds) are left out on purpose, since

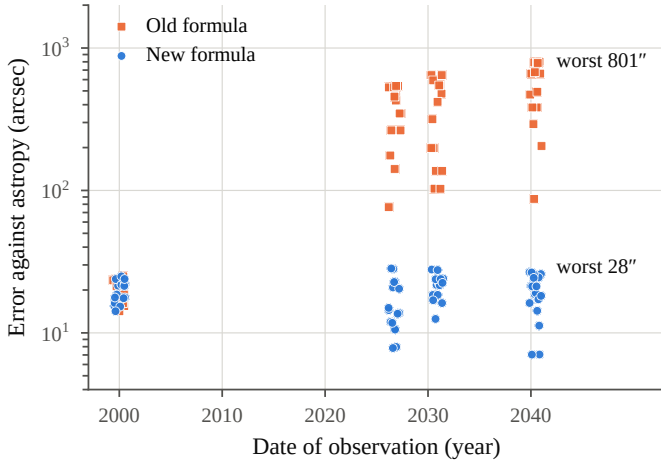


Fig. 2. Pointing error against astropy for the 66 reference cases (12 stars, 3 sites, 4 dates), old and new formula

the narrowest eyepiece field in use is at least 15 arcminutes across. The worst of the 66 cases is now 28 arcseconds, and the unit tests fail if either correction is applied twice. The precession matrix is cached per time step, because the sky view converts thousands of stars in each frame. With the correction, the error stays between 7 and 28 arcseconds at every date, about the size of the nutation and aberration that the model omits.

E. Alignment and guidance

The phone’s rotation-vector sensor gives the orientation of the phone, from which the app derives the direction of the telescope axis. The observer centres a bright star in the eyepiece and selects it. The app then computes a rotation that moves its predicted pointing onto the star, in azimuth first and then in altitude, following AstroHopper [4]. For a chosen target the app shows the remaining altitude and azimuth, or right ascension and declination for an equatorial mount, together with a bullseye that fills when the target lies inside the eyepiece’s true field. The true field is computed from the eyepiece’s apparent field and the focal lengths of the eyepiece and the telescope.

A revised alignment flow is implemented on the development branch. On first launch it asks for the telescope type, mount and phone placement. To align, the observer centres the chosen star in the eyepiece, drags the map until the star sits under a fixed centre marker, and confirms; the app then computes the rotation exactly and checks that the star lands on the telescope axis. A second star can be added to check and refine the alignment. Section V lists what of this has been tested.

F. On-device plate solver

The solver is about 1,200 lines of Kotlin with no native code. It first estimates the image background on a coarse grid and the noise from the median absolute deviation. It then marks pixels about five standard deviations above the

TABLE II
PLATE SOLVER ON SYNTHETIC IMAGES (JVM, DESKTOP CPU). TIMES COME FROM ONE RUN AND VARY BETWEEN RUNS

Case	Solved	Centre error	Time
Wide $60^\circ \times 45^\circ$, blind	5/5	4–11"	0.15–0.32 s
Wide, hint 15° off	5/5	1–8"	0.16–0.42 s
Wide, mirrored, blind	5/5	3–11"	0.16–0.24 s
Wide, 1–2% lens distortion	4/4	15–44"	0.16–0.52 s
12° field, blind	2/2	0.7–1.1"	0.17–0.31 s
5° field, hinted	3/3	0.1–0.6"	0.28–0.55 s
1° eyepiece, hint 1° off	5/5	0.1–0.5"	0.17–0.26 s
Negative cases (28)	0 wrong	n/a	0.10–3.1 s

background and groups them into connected blobs. Each star position is a flux-weighted centroid, and its elongation comes from second moments. Single hot pixels and saturated regions are discarded. The image is rejected as too bright, trailed or nearly empty before any matching is tried, and the user is told which case applies.

The pixel scale is known within a range from the telescope and eyepiece settings or from the phone camera’s focal length. The solver therefore matches triangles of the brightest detected stars against catalogue triangles using real angular side lengths. The magnitude limit is chosen so that the catalogue offers about one to three times as many stars as were detected. Each candidate match gives a similarity transform with a possible mirror flip in a gnomonic projection. The solver projects all catalogue stars in the field, counts detections that land within the centroid tolerance, and refines the fit by least squares.

A wrong answer would send the observer to the wrong part of the sky, so acceptance is strict. A solution needs at least six matched stars spread over at least a quarter of the frame, and a scale inside the hinted range. It also needs a false-alarm probability below 10^{-6} after multiplying by the number of hypotheses tried. The false-alarm probability is the binomial chance that random stars at the observed density would match as many detections. Real solutions in the tests reached probabilities between 10^{-20} and 10^{-120} .

Table II summarises the synthetic evaluation. The test images are rendered from the same catalogues with Gaussian star profiles, Poisson and read noise, a sky gradient, hot pixels, 10% of stars removed and a few false stars added. Eyepiece images black out everything outside a 1-degree circle, as a phone held to an eyepiece records it. All 29 positive cases were solved. Every one of the 28 negative cases (blank, noise, saturated and trailed images, wrong scale hints, and random-star fields) returned a failure with the matching reason, so none of the 57 test calls gave a wrong answer. Figure 3 plots the centre error and solve time of each solved case. Errors of 0.1 arcsecond are the resolution of the printed test output.

The sparsest eyepiece test field holds eight catalogue stars and solves with six matches, the minimum the acceptance test allows. Eyepiece fields with fewer than six stars brighter than magnitude 10.5 cannot be solved, and the solver returns “no match” for them.

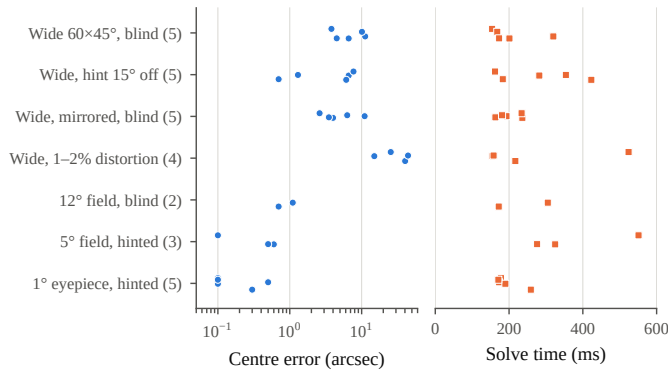


Fig. 3. Centre error and solve time for the 29 solved synthetic images, grouped as in Table II. The number of images in each group is in brackets

G. Interface

The sky view follows Stellarium’s look, with an atmosphere, the Milky Way, landscapes, light pollution on the Bortle scale, constellation art and star colours. It can also draw the equatorial grid, ecliptic, meridian and constellation boundaries. A night mode draws everything in dim red to preserve dark adaptation. Object pages show a rise, transit and set summary and an altitude graph for the night with twilight shading. An Events screen lists meteor showers, conjunctions, eclipses, lunar occultations and satellite passes computed on the phone. An offline voice assistant answers spoken questions in English and Hindi through Android’s speech recogniser, and the whole interface is available in Hindi.

H. Verification and release

The astronomy layer has JVM unit tests: golden values from the original web code, the astropy reference cases, the plate-solver suite and the mounting geometry, 85 tests in all at the time of writing. The importer has 19 tests. The interface is tested in a Compose Desktop harness that drives the real screens with simulated touch at a 360 by 780 dp phone size. With the setup wizard and the revised alignment flow it runs 82 journey, flow and stress tests. An automated audit also renders 440 screen variants (English and Hindi, day and night, two widths) and checked touch-target size, clipped or overlapping text and contrast, with no findings. Every push to the repository runs the tests, builds the APK, checks that the sky data is inside it and scans the history for leaked secrets with gitleaks. A build from the main branch is published as a downloadable APK.

IV. SYSTEM DESIGN

This section describes the design with UML diagrams drawn from the source code. The diagrams are generated from PlantUML files kept next to the paper. Where the code is unfinished, the figure says so.

A. Use cases

Figure 4 shows what an observer does with the app. Dashed ovals mark use cases whose current form is in development:

the revised alignment flow, the setup wizard with phone placement, and plate solving with the camera. The solver itself is written and tested, but the camera capture code is not. The other use cases are in the current build. Following guidance includes choosing a target, and a plate solve extends alignment because a solved image is meant to be applied to the alignment.

B. Structure

Figure 5 shows the runtime structure. `MainActivity` is the only code that touches the Android framework. It reads the rotation-vector sensor, gets a location fix, runs speech recognition and speech output, stores settings in shared preferences, loads the assets, and downloads satellite orbit elements from CelesTrak when a network is available. It passes the sensor matrix and settings to `SkyState` and starts `SkyScreen`. The `ui` package has no Android imports and `astro` does not depend on `ui`, which is what lets the JVM tests and the desktop harness load each layer alone. `PlateSolver` and `SolverStars` are in `astro` but no screen calls them yet.

Figure 6(a) gives the core classes. `SkyState` holds what the screen shows: time, place, phone rotation, pointing mode, alignment state and matrix, the target and the eyepiece settings. Its guidance methods combine the camera frame from `Pointing` with the ray to the target. `Catalog` owns the `SkyObject` list and answers cone, name and constellation queries. The members below the dotted line in `SkyState`, and `TelescopeSetup` with its enumerations, exist only in the development branch.

Figure 6(b) shows the solver classes. `PlateSolver` has two entry points, `detectStars` and `solve`. The `StarSource` interface lets the same solver run on the bright catalogue through `CatalogStars`, on the 580k-star file through `SolverStars`, or on a plain list in tests. The result is a sealed class: `Solved` carries the pose and the false-alarm probability, and `Failed` carries one of four reasons.

C. Alignment and plate solving

Figure 7 shows the alignment states of the revised flow. In the current build, tapping a star while picking aligns at once, as if the telescope were already centred. The revised flow inserts a centring state, so nothing is calibrated until the observer confirms. Cancelling returns to the aligned state if a calibration exists and to the unaligned state otherwise. The calibration is cleared when the phone mounting changes or the settings are reset.

Figure 8 follows one alignment through the classes. The observer starts alignment and taps a star. `SkyState` accepts it unless it is the Sun, the Moon, not a star or planet, or below the horizon. Dragging the map only changes a pending adjustment, so browsing cannot change the calibration. Confirming computes `alignMatrix` from the uncorrected camera frame and the star, and rejects it if the corrected axis does not land on the star.

Figure 9 shows the plate solve as an activity. The three early checks reject a bad image before any matching starts

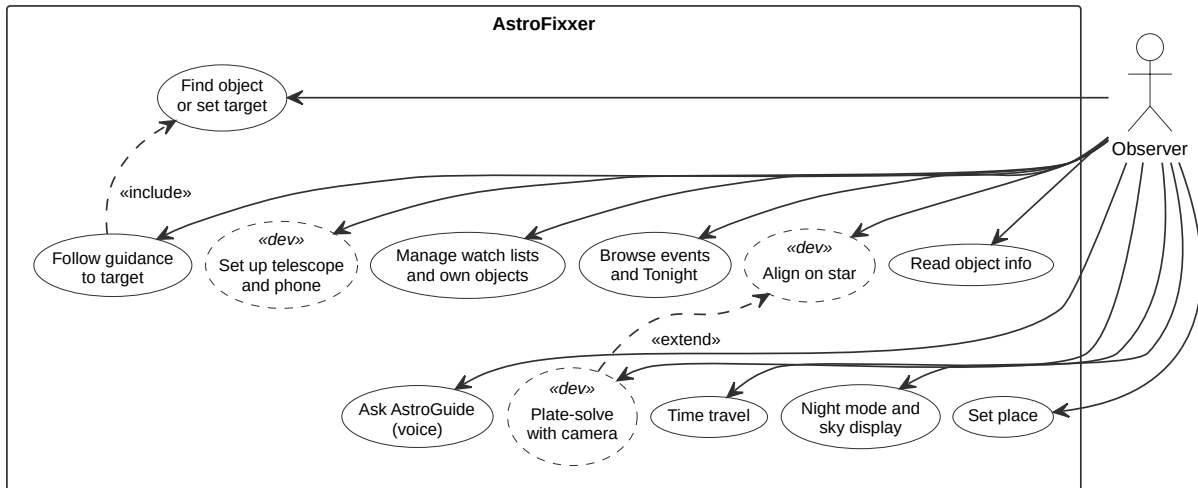


Fig. 4. Use cases of the observer. Dashed ovals are in development

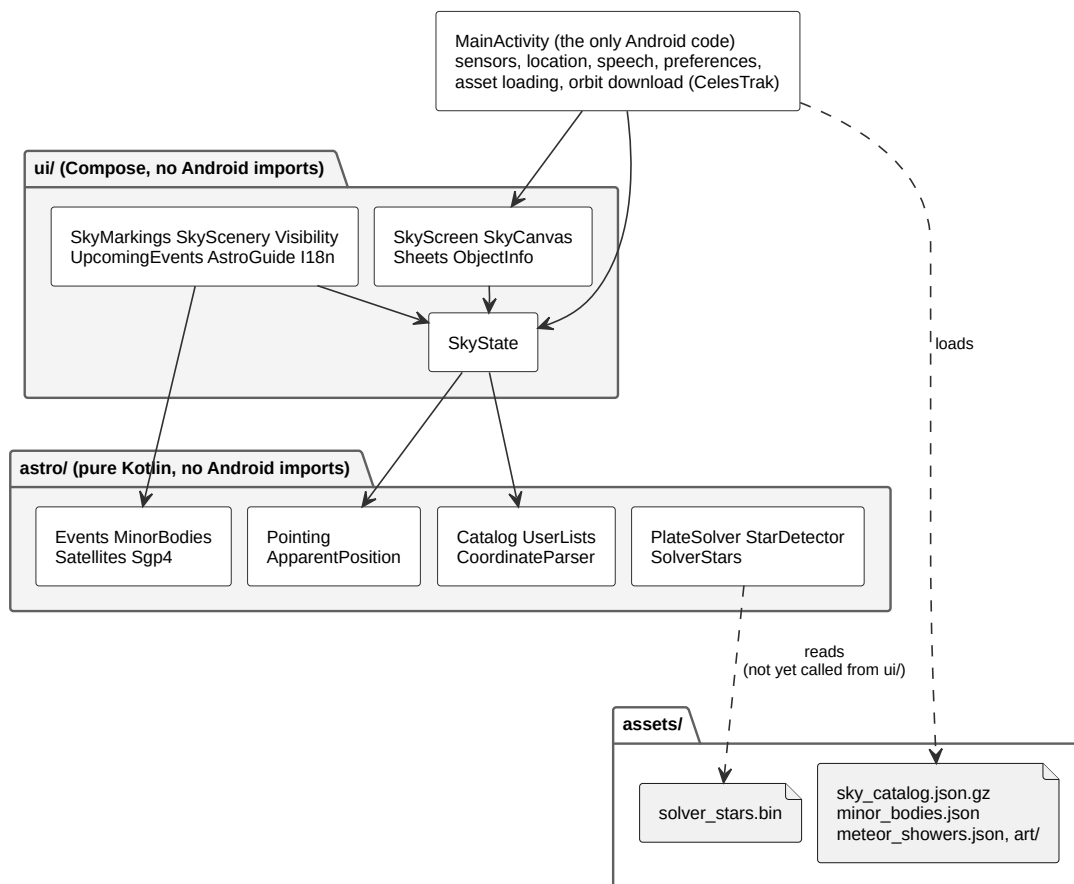


Fig. 5. Components of the app and the assets they read. Dashed arrows are data access

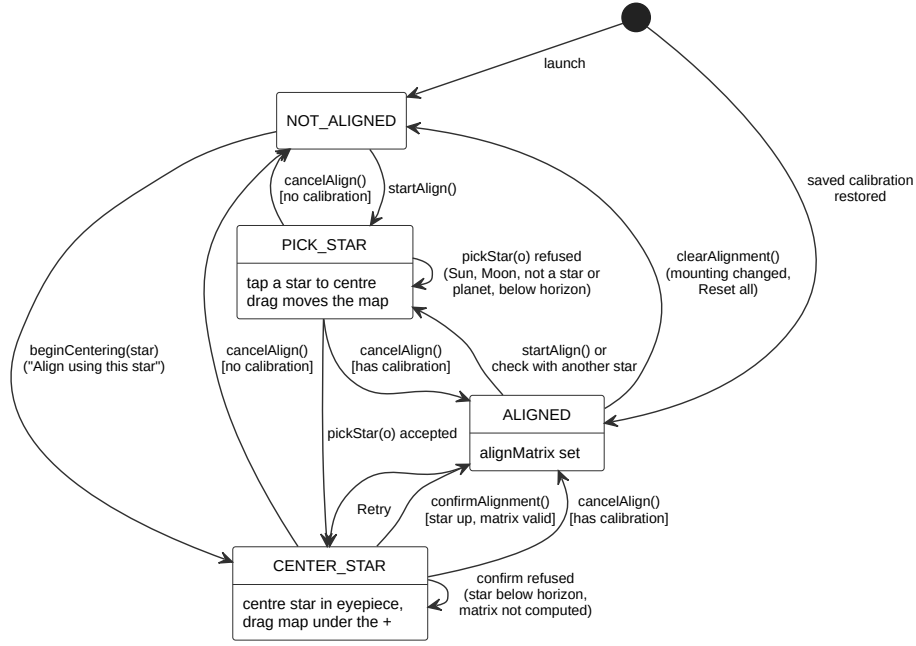


Fig. 7. Alignment states and transitions, from SkyState (in development)

and tell the observer which problem was found. Matching then builds hypotheses from triangles, and each hypothesis is verified against the catalogue. A result is accepted only if it passes the acceptance test described in Section III-F; otherwise the solver reports no match.

D. Build, test and release

Figure 10 shows how the code becomes an installed app. The developer runs the importer to write the data assets and pushes the source. Three GitHub Actions jobs run on every push: `build` runs the JVM tests, assembles the APKs and checks that the sky data is inside the package; `ui-check` runs the desktop tests and saves screenshots; and `secret-scan` runs gitleaks over the whole history. A push to the main branch also publishes the latest APK as a GitHub release, and a version tag starts a separate workflow that builds a signed bundle and APK. The phone downloads the APK and needs no network afterwards, except to refresh satellite orbits.

E. Development process

Figure 11 lists the phases the project followed. Work started from an analysis of the web application and a plan, and the plan went through a review that climbs a ladder of questions (does it need to exist, is it already in the code, does the platform provide it) before any code is written. Each change ends with tests, an entry in the handoff log and a push that starts the CI jobs above. The current phase, 7b, adds the precession fix, the plate solver and the setup and alignment work; the last of these is still open, and the field test is written but has not been run.

V. REVIEW AGAINST RELATED WORK

A. Where AstroFixxer differs

Against AstroHopper and SkEye, AstroFixxer adds a way to measure the pointing instead of trusting the sensors after one alignment. Against StarSense Explorer and PiFinder, it needs no dock, mirror or separate computer; the phone already on the tube does the work. Unlike astrometry.net, the solver does not attempt blind solving at every scale. It uses the scale implied by the observer's equipment and the rough pointing from the sensors, which keeps the star file to 2.94 MB and a hinted solve to about half a second on a desktop CPU. The catalogue is larger than AstroHopper's and is fully offline, which matters at dark sites without mobile coverage. The Indian sky culture and the Hindi interface have no counterpart in the other free tools reviewed here.

B. Limitations and what is not yet tested

The evidence in Section III comes from unit tests, synthetic images and a desktop test harness. None of it replaces a night at a telescope, and the following points remain open.

- The plate solver has only seen rendered star fields. Real phone photographs have non-Gaussian star images, compression artefacts, lens distortion beyond the 2% tested, and noise that depends on the phone's processing. The detection thresholds were tuned on synthetic data.
- Solving through an eyepiece depends on the phone recording stars of magnitude 9 to 10 in a short exposure. This has not been measured.
- The revised alignment flow has only run in the desktop test harness, and the camera capture code is still under development. Neither has run on a phone.

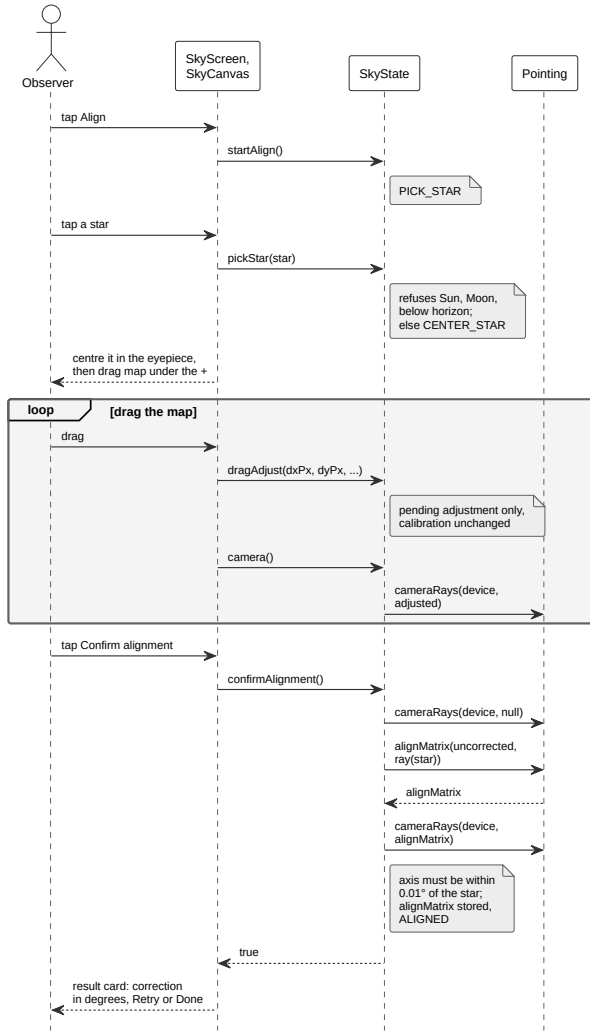


Fig. 8. Star alignment with drag to align (in development)

- The coordinate model omits nutation and aberration. At 28 arcseconds this is well inside an eyepiece field, but it would matter for setting-circle readouts on a precisely polar-aligned mount.
- Sensor drift between alignments has not been measured on real phones or near real tubes.

A field protocol is already written: three phones (with and without a magnetometer), two telescopes, and ten Messier objects reached from alignment stars 5 to 20 degrees away, recording the time to target and the misses.

VI. CONCLUSION

Free push-to apps put telescope guidance on a phone but cannot check where the telescope points. Camera-based finders can, but only with extra hardware. AstroFixxer brings the two together on an unmodified Android phone: sensor guidance from AstroHopper, an offline Stellarium-derived catalogue, a coordinate model accurate to 28 arcseconds against astropy, and a Kotlin plate solver that found every synthetic test pointing and accepted no false match. The next step is the

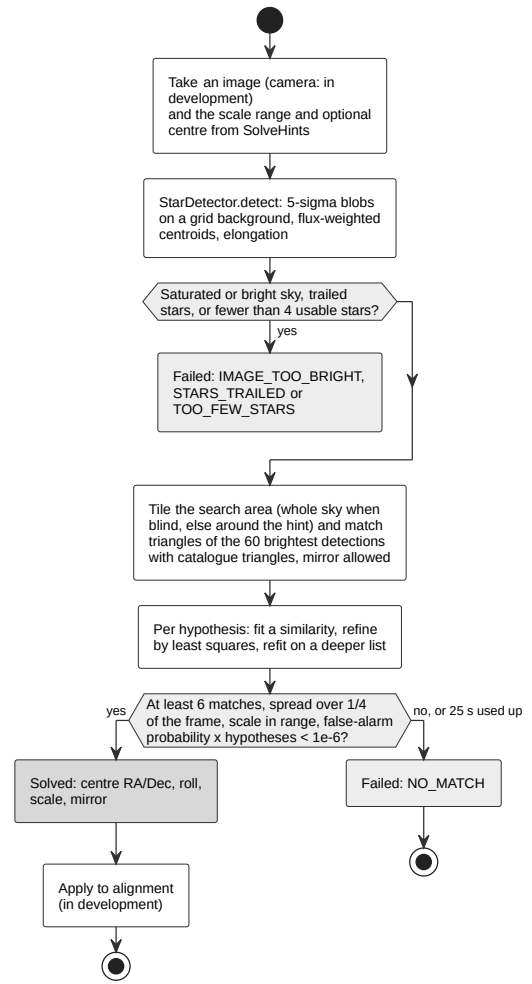


Fig. 9. Plate solve, from PlateSolver and StarDetector. Camera capture and applying the result to the alignment are in development

field test described in Section V, run with real photographs from phones at the eyepiece and beside the tube.

ACKNOWLEDGMENT

AstroFixxer is built on the work of others. Artyom Beilis wrote AstroHopper [4], whose pointing code and licence AstroFixxer inherits, and the AstroFixxer web application [12] was the starting point for the Android port. All sky data comes from the Stellarium project [7]. The solver's star list derives from ESA's Gaia DR3 [17] and Hipparcos [18] catalogues as packaged by Stellarium, and the bright-star catalogue from the HYG database [14]. Greg Miller's public-domain VSOP87 series and CPReduce code supply the planet positions, NASA's Goddard Space Flight Center publishes the eclipse tables, and CelesTrak provides the satellite orbits. Table III lists every third-party component in the project and its licence.

REFERENCES

- [1] Simulation Curriculum Corp., "SkySafari scope setup help," <https://support.simulationcurriculum.com/hc/en-us/articles/115015943188-Scope-Setup-Help>, accessed Sep. 30, 2026.

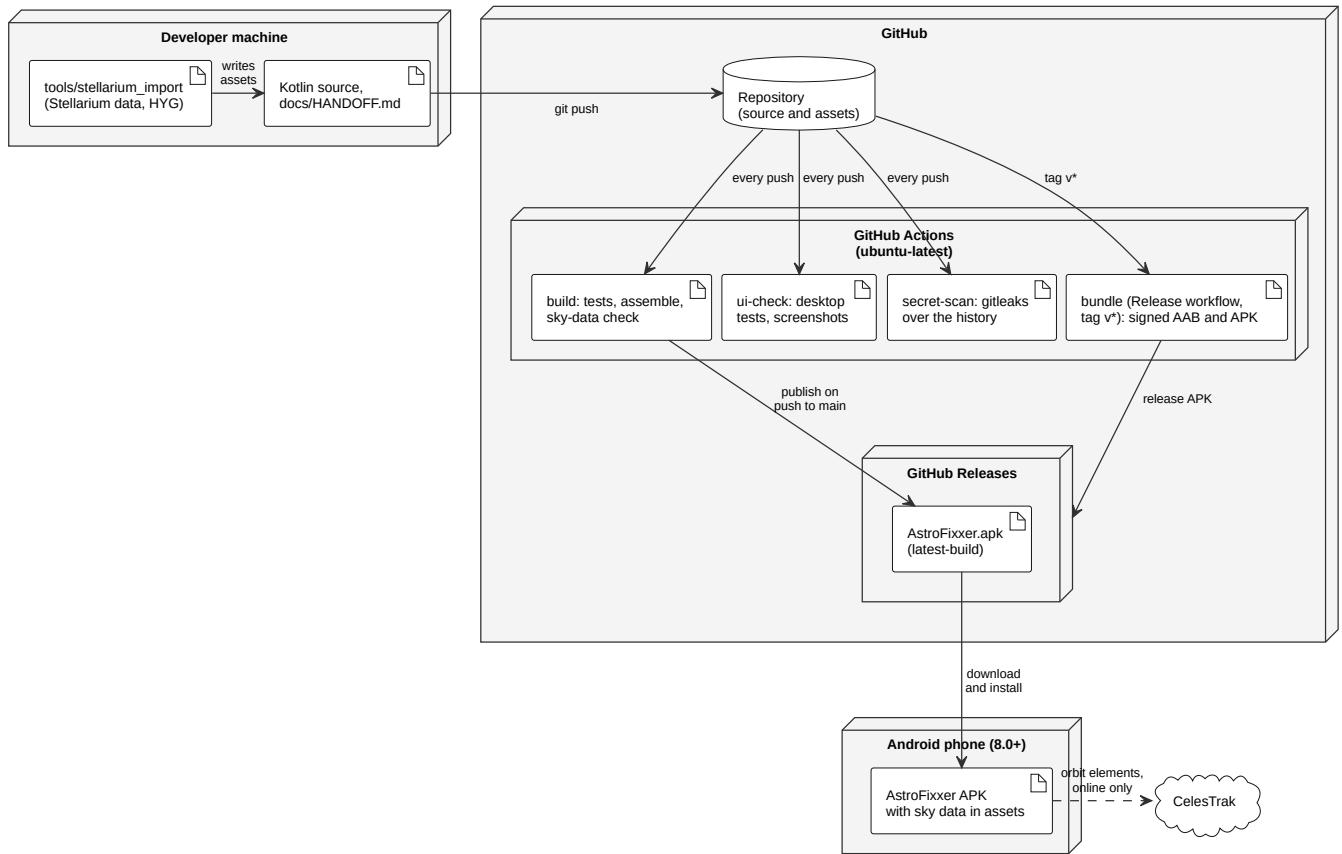


Fig. 10. Deployment: from the developer machine through GitHub Actions to the phone

- [2] Celestron, “StarSense Explorer technology,” <https://www.celestron.com/pages/starsense-explorer-technology>, accessed Sep. 30, 2026.
- [3] PiFinder project, “PiFinder: A plate solving telescope finder based around a Raspberry Pi and RPi HQ camera,” GitHub repository, <https://github.com/brickbots/PiFinder>; documentation at <https://pifinder.readthedocs.io>, accessed Sep. 30, 2026.
- [4] A. Beilis, “AstroHopper: Digital setting circles implemented in a smartphone,” GitHub repository, <https://github.com/artiom-beilis/skyhopper>, 2025, gPLv3. Accessed Sep. 30, 2026.
- [5] “SkEye: Advanced planetarium for Android,” <https://skeye.rocks/apps/skeye/index.html>, accessed Sep. 30, 2026.
- [6] Sky Map Devs, “Sky map (formerly Google sky map),” GitHub repository, <https://github.com/sky-map-team/stardroid>, accessed Sep. 30, 2026.
- [7] G. Zotti, S. M. Hoffmann, A. Wolf, F. Chéreau, and G. Chéreau, “The simulated sky: Stellarium for cultural astronomy research,” *Journal of Skyscape Archaeology*, vol. 6, no. 2, pp. 221–258, 2021.
- [8] D. Lang, D. W. Hogg, K. Mierle, M. Blanton, and S. Roweis, “Astrometry.net: Blind astrometric calibration of arbitrary astronomical images,” *The Astronomical Journal*, vol. 139, no. 5, pp. 1782–1800, 2010.
- [9] J. Brown, K. Stubis, and K. Cahoy, “TETRA: Star identification with hash tables,” in *Proc. 31st Annual AIAA/USU Conference on Small Satellites*, Logan, UT, USA, 2017.
- [10] H. Kleijn, “ASTAP, astrometric STACKing program,” <https://www.hnsky.org/astap.htm>, accessed Sep. 30, 2026.
- [11] R. Marbel, B. Ben-Moshe, and R. Yozevitch, “Star-tracker algorithm for smartphones and commercial micro-drones,” *Sensors*, vol. 20, no. 4, p. 1106, 2020.
- [12] AadidevRaizada, “AstroFixxer web application,” GitHub repository, <https://github.com/AadidevRaizada/AstroFixxer>, 2026, accessed Sep. 30, 2026.
- [13] “AstroFixxer for Android,” GitHub repository, <https://github.com/Zenithquonta/astrofixxer-android>, 2026, gPLv3. Accessed Sep. 30, 2026.
- [14] D. Nash, “The HYG database,” GitHub repository, <https://github.com/astronexus/HYG-Database>, version 3, CC BY-SA. Accessed Sep. 30, 2026.
- [15] J. H. Lieske, T. Lederle, W. Fricke, and B. Morando, “Expressions for the precession quantities based upon the IAU (1976) system of astronomical constants,” *Astronomy and Astrophysics*, vol. 58, pp. 1–16, 1977.
- [16] Astropy Collaboration, “The Astropy project: Sustaining and growing a community-oriented open-source project and the latest major release (v5.0) of the core package,” *The Astrophysical Journal*, vol. 935, no. 2, p. 167, 2022.
- [17] Gaia Collaboration, A. Vallenari *et al.*, “Gaia data release 3: Summary of the content and survey properties,” *Astronomy and Astrophysics*, vol. 674, p. A1, 2023.
- [18] M. A. C. Perryman, L. Lindegren, J. Kovalevsky *et al.*, “The HIPPARCOS catalogue,” *Astronomy and Astrophysics*, vol. 323, pp. L49–L52, 1997.
- [19] P. Bretagnon and G. Francou, “Planetary theories in rectangular and spherical variables: VSOP87 solutions,” *Astronomy and Astrophysics*, vol. 202, pp. 309–315, 1988.
- [20] D. A. Vallado, P. Crawford, R. Hujsak, and T. S. Kelso, “Revisiting Spacetrack Report #3,” in *Proc. AIAA/AAS Astrodynamics Specialist Conference*, Keystone, CO, USA, 2006, AIAA 2006-6753.

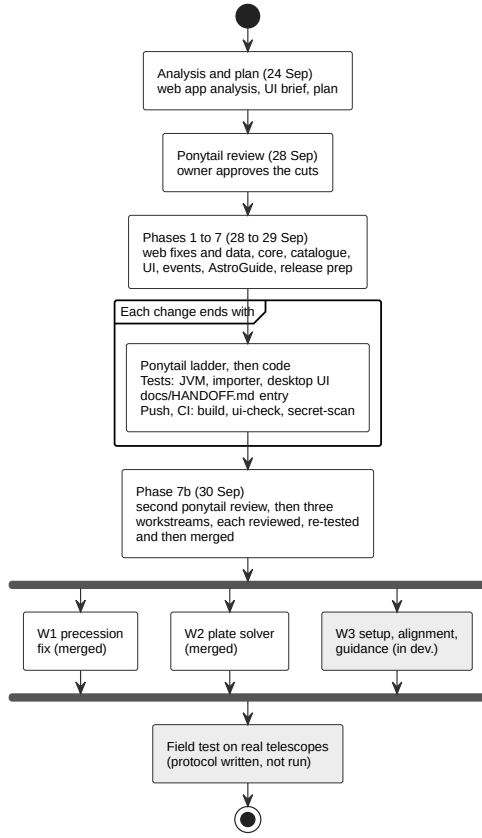


Fig. 11. Development phases and the steps that end each change

TABLE III
THIRD-PARTY REPOSITORIES, DATA AND LIBRARIES

Component	Licence
AstroHopper / skyhopper (A. Beilis)	GPLv3
AstroFixxer web app (upstream)	GPLv3
Stellarium deep-sky catalogue, names, meteor showers, minor-body orbits, star catalogues	GPL-2.0-or-later
Stellarium sky cultures (modern, Indian)	CC BY-SA 4.0
Stellarium constellation illustrations	Free Art License
Gaia DR3 (ESA), via Stellarium	CC BY-SA 3.0 IGO
Hipparcos (ESA), via Stellarium	ESA
HYG v3 star database	CC BY-SA
VSOP87 Java series and CPReduce (G. Miller)	Public domain
NASA eclipse tables (F. Espenak, NASA GSFC)	Public domain
CelesTrak orbital elements	CelesTrak terms
Kotlin, Jetpack Compose, AndroidX Activity	Apache 2.0
Compose Multiplatform (desktop test harness)	Apache 2.0
Gradle build tool	Apache 2.0
JUnit 4	EPL 1.0
Robolectric android-all (compile check)	Apache 2.0
JSON-java (org.json)	See project
astropy (reference values only) [16]	BSD 3-Clause
gitleaks (secret scanning in CI)	MIT